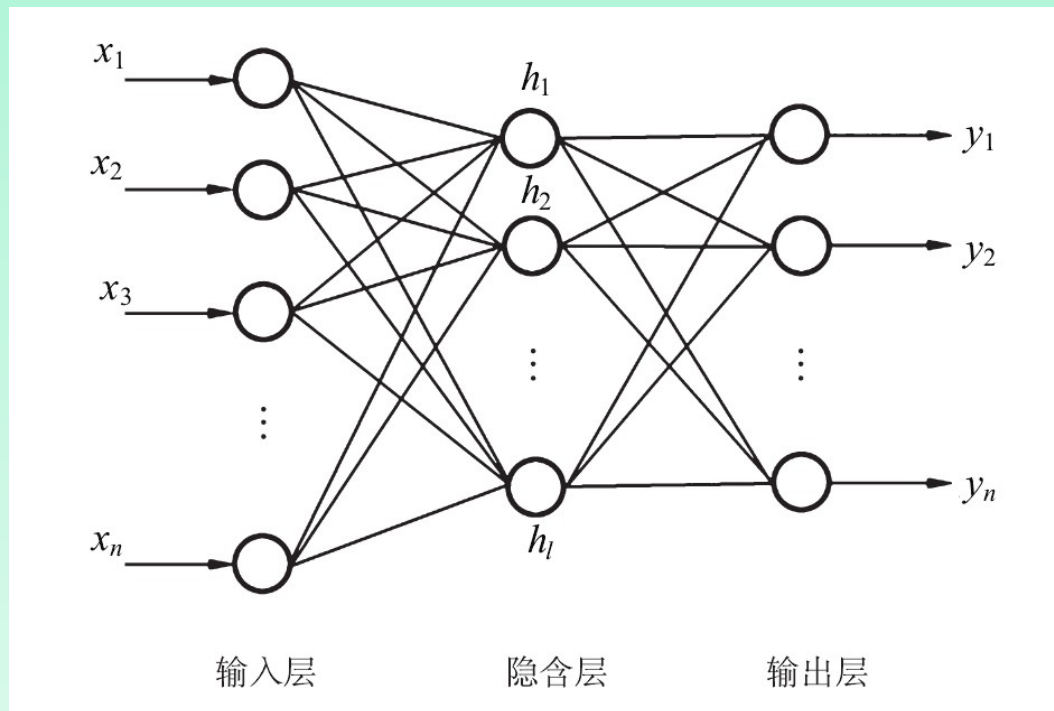


机器学习

第4章 人工神经网络 (ANN)

4.5 多层网络和反向传播算法

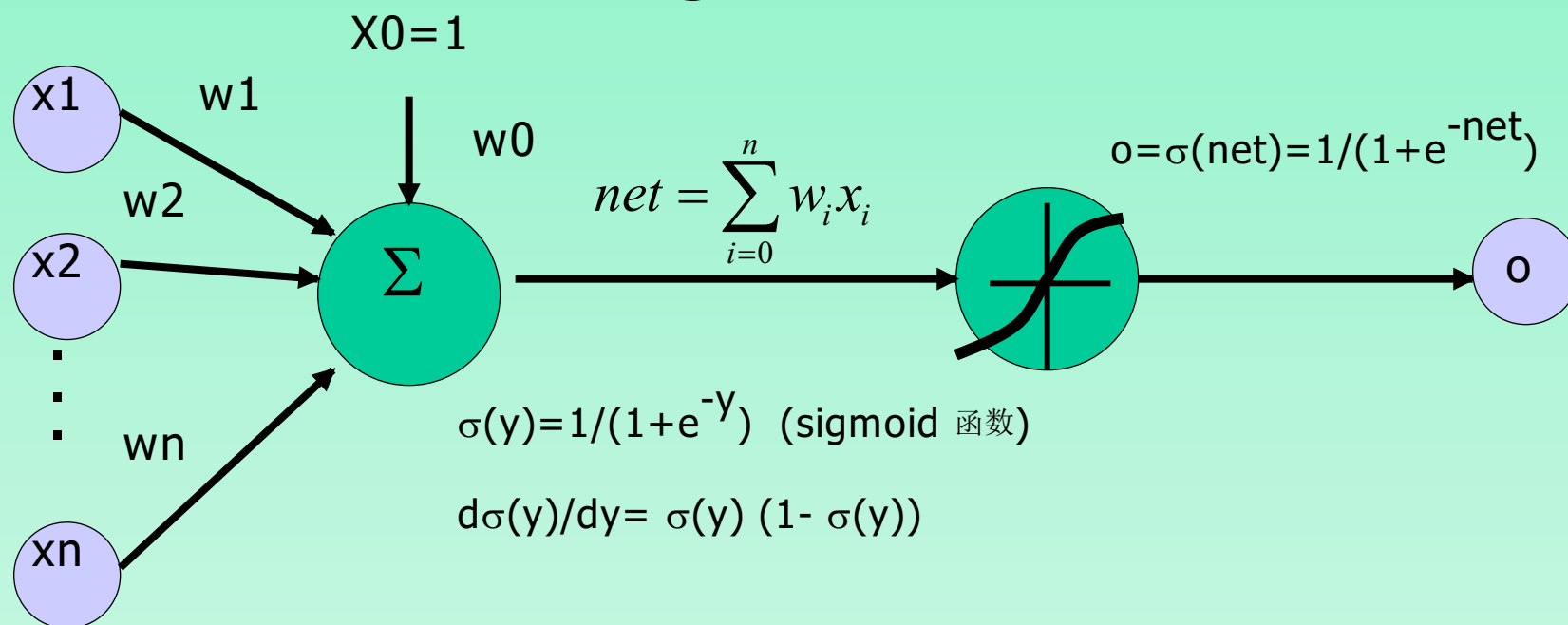
- 多层网络能够表示种类繁多的非线性决策面
- 如何构建多层网络？



4.5.1 可微阈值单元

- 构建多层网络需满足：
 - 能表示高度非线性的函数
 - 适用于梯度下降算法（可微分）
- 构建多层网络的神经元应该满足
 - 输出是输入的非线性函数
 - 输出是输入的可微函数

Sigmoid 单元



sigmoid 神经元的梯度下降学习规则:

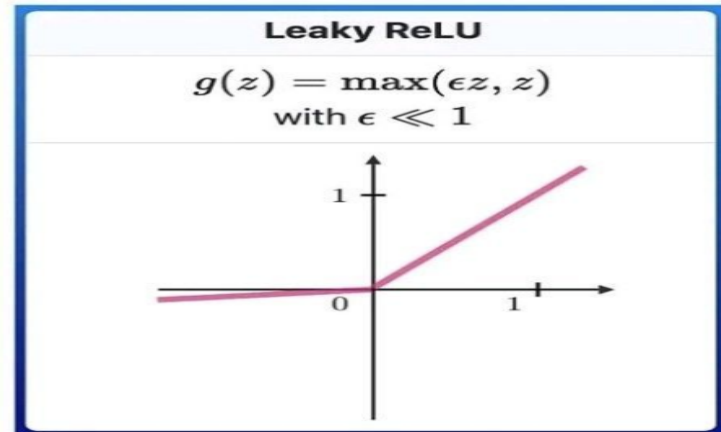
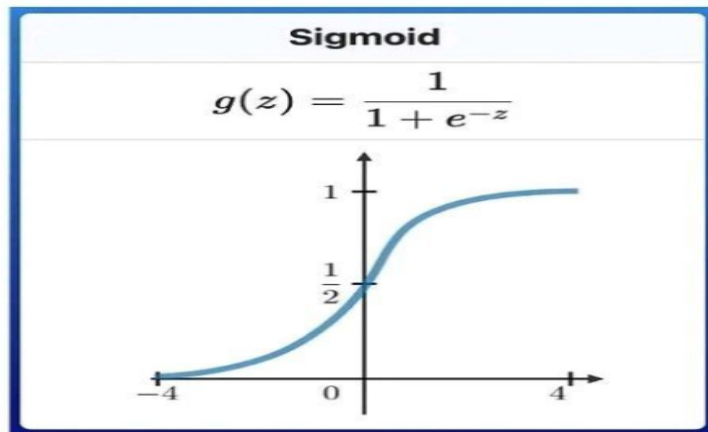
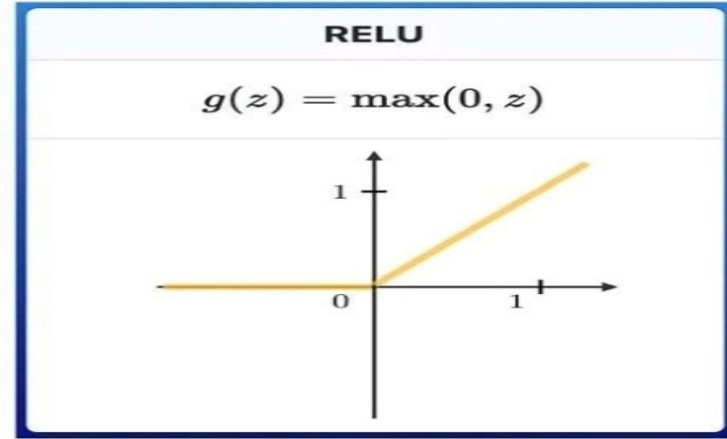
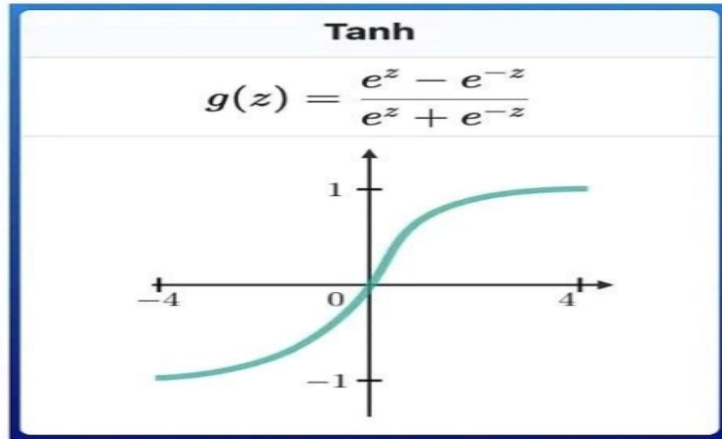
$$\frac{\partial E}{\partial w_i} = -\sum_{\epsilon} d_{\epsilon} (1 - o_{\epsilon}) o_{\epsilon} x_i$$

- Sigmoid 单元的特点
 - 平滑的可微阈值函数
 - Sigmoid 函数（logistic函数, Squashing函数）
 - 输出范围为0到1
 - 单调增加
 - 它的导数很容易用输出来表示

$$\sigma'(x) = \frac{1}{(1+e^{-x})} \left(1 - \frac{1}{(1+e^{-x})} \right)$$

$$\sigma'(x) = \sigma(x) \cdot (1 - \sigma(x))$$

- 其他神经元活跃函数
 - sigmoid 的陡峭性(e^{-ky})



- 神经元活跃函数比较
 - Sigmoid, Tanh: 当输入很大或很小时, 梯度趋于零, 导致梯度消失
 - Relu的非饱和性可以有效地解决梯度消失问题
 - Relu计算复杂度低
 - Relu的局限性在于训练过程中会导致神经元死亡, 可用Leaky Relu神经元

$$\frac{d}{dx} \tanh(x) = 1 - \tanh(x)^2$$

the derivative of the ReLU function as

$$f'(x) = \begin{cases} 1 & \text{if } x > 0 \\ 0 & \text{if } x \leq 0 \end{cases}$$

$$\frac{d}{dx} \sigma(x) = \sigma(x) \cdot (1 - \sigma(x))$$

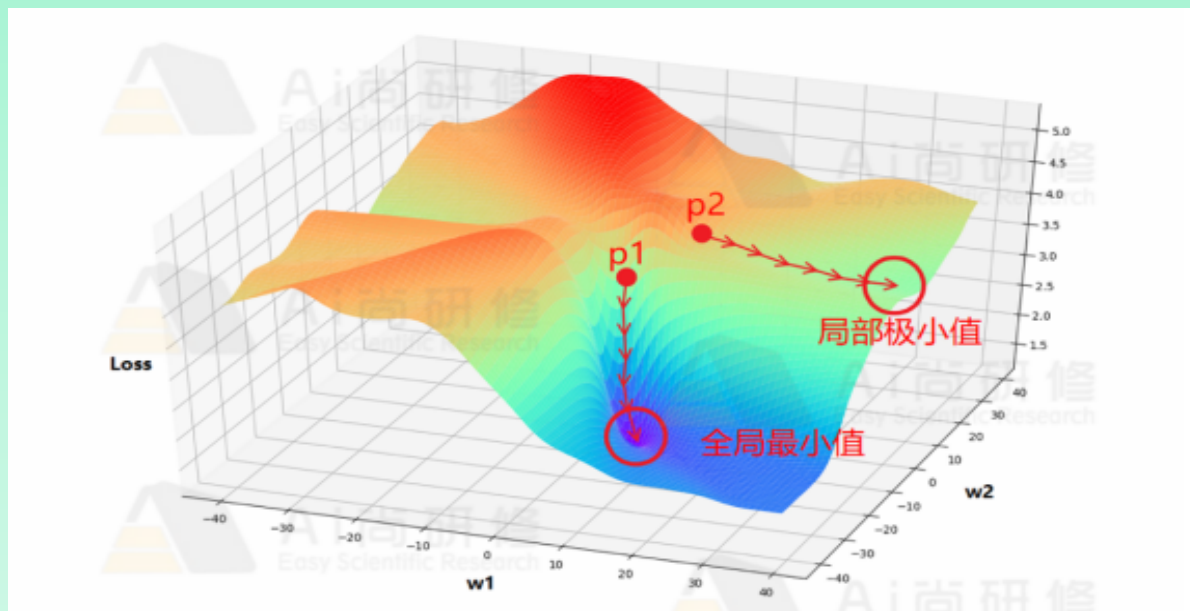
$$f'(x) = \begin{cases} \alpha & \text{for } x < 0 \\ 1 & \text{for } x \geq 0 \end{cases}$$

4.5.2 反向传播算法

- 学习多层网络的权重
 - 采用梯度下降，使网络输出值和目标值之间的平方误差最小化
 - 重新定义E，对所有网络输出单元上的误差求和

$$E(\vec{w}) = \frac{1}{2} \sum_{d \in D} \sum_{k \in \text{outputs}} (t_{kd} - o_{kd})^2$$

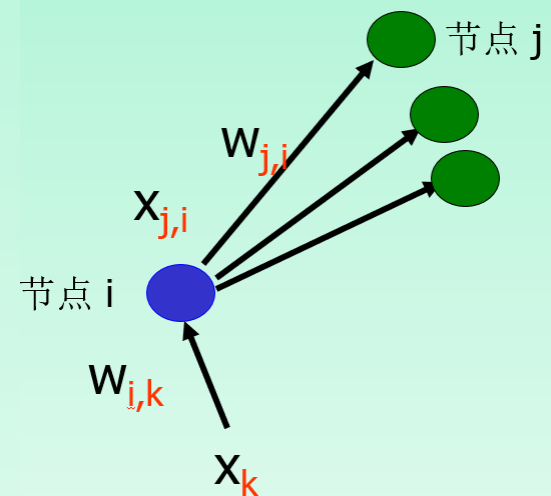
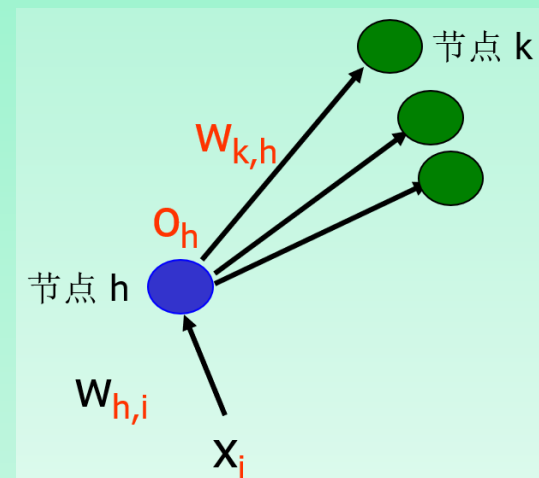
- 反向传播算法的难点
 - 搜索巨大的假设空间
 - 误差面可以有多个局部极小值



- 实践中发现反向传播算法通常能搜索到较佳结果

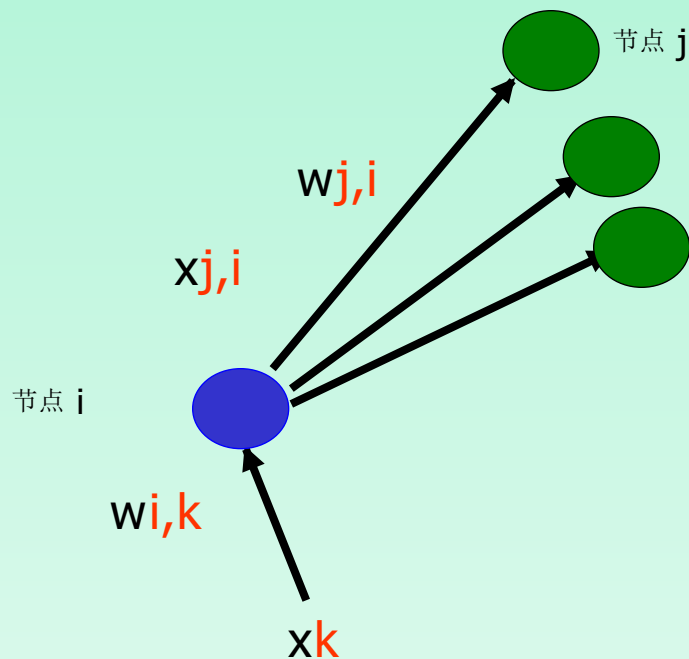
随机梯度下降反向传播算法（单隐藏层）

- 将每个 w_i 初始化为一些小的随机值
 - 在满足终止条件之前，执行
 - 对于每个训练样例 $\langle (x_1, \dots, x_n), t \rangle$ 执行
 - 把实例 $(x_1, \dots, x_n)$ 输入网络, 计算网络输出 o_k
 - 对于每个输出层单元 k
 - $\delta_k = o_k(1-o_k)(t_k-o_k)$
 - 对于每个隐藏单元 h
 - $\delta_h = o_h(1-o_h) \sum_{k \text{ in outputs}} w_{k,h} \delta_k$
 - 更新每个网络权值 $w_{j,i}$
 - $w_{j,i} = w_{j,i} + \Delta w_{j,i}$ 其中
 - $\Delta w_{j,i} = \eta \delta_j x_{j,i}$
- (注: $x_{j,i}$: 单元 i 的输出, 对应于 $w_{j,i}$)



- 网络符号表示:

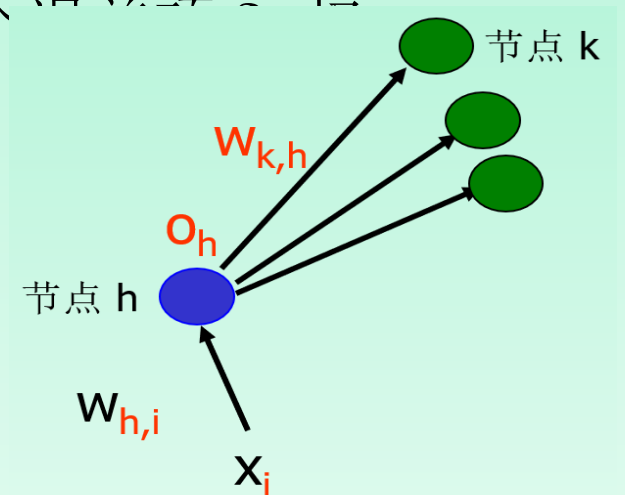
- 每个结点被赋予一个序号, 这里的“结点”要么是网络的输入, 要么是网络中某个单元的输出。
- x_{ji} 表示结点*i*到单元*j*的输入
- w_{ji} 表示对应的权值
- δ_n 表示与单元*n*相关联的误差项



- 算法解释
 - 网络结构
 - 前馈网络，包含两层sigmoid单元
 - 每层的单元连接到上一层的所有单元
 - 学习方法
 - 增量或者随机梯度下降算法
 - 权值更新($\Delta w_{j,i} = \eta \delta_j x_{j,i}$, 比较delta法则: $\Delta w_i = \eta(t - o)x_i$)
 - 学习速率 η
 - $w_{j,i}$ 上的输入值 x_{ji}
 - 单元j的误差项($\delta_j = -\frac{\partial E}{\partial net_j}$)

- 理解 δ_j ($\Delta w_{j,i} = \eta \delta_j x_{j,i}$)
 - 对于输出单元k, δ_k
 - δ_k 是 $(t_k - o_k)$, 乘以系数 $o_k(1 - o_k)$, (sigmoid 函数的导数)
 - $\delta_k = o_k(1 - o_k)(t_k - o_k)$
 - 对于隐藏单元h, δ_h
 - 缺少直接的目标值来计算隐藏单元的误差值。
 - 受隐藏单元h影响的每一个下游单元的误差 δ_k 进行加权求和来计算误差, 每个权重为 w_{kh}

$$\delta_h = o_h(1 - o_h) \sum_{k \text{ in outputs}} w_{k,h} \delta_k$$



- 采用两种不同的梯度下降算法修改权值：
 - 真实梯度：
 - 计算所有样例关于E的梯度，对所有训练样例的 $\Delta w_{j,i} = \eta \delta_j x_{j,i}$ 求和，并修改权值
 - 随机梯度：
 - 单样本修改
- BP算法终止条件
 - 迭代次数达到固定值。
 - 误差降到某个阈值以下。
 - 在独立的验证样例集合上，误差符合某个标准。
 - 终止标准很重要：误差过大或过拟合

4.5.2.1 增加冲量项



- 反向传播算法的变体

- 第 n 次迭代权值的增量依赖于发生在第 $n-1$ 次迭代时的更新

$$\Delta w_{ji}(n) = \eta \delta_j x_{ji} + \alpha \Delta w_{ji}(n-1)$$

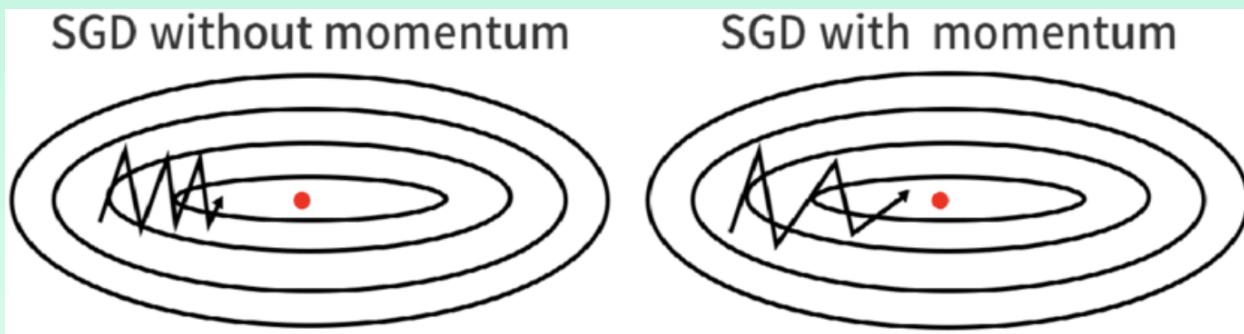
$0 \leq \alpha < 1$, $\Delta w_{ji}(n-1)$ 被称为冲量项

$$v_{t+1} = \beta v_t + (1 - \beta) \nabla L(w_t)$$

$$w_{t+1} = w_t - \eta v_{t+1}$$

- 理解冲量项

- 冲量项可以使球沿着误差曲面的同一个方向滚动
 - 影响1: 冲量项会使球滚过局部极小值或平坦的区域
 - 影响2: 在梯度方向不变的区域逐渐增大搜索步长的效果, 从而可以加快收敛



4.5.2.2 学习任意的无环网络

- 学习算法推广到任意深度的前馈网络。
 - 第m层的单元r的 δ_r 值由更深的m+1层 δ_s 值计算

$$\delta_r = o_r(1 - o_r) \sum_{s \in \text{layer } m+1} w_{sr} \delta_s$$

- 算法推广到任何有向无环结构，计算任意内部单元r的误差项:

$$\delta_r = o_r(1 - o_r) \sum_{s \in \text{Downstream}(r)} w_{sr} \delta_s$$

Downstream(r) 是在网络中单元r的直接的下游单元的集合，或者说输入中包括r的输出的所有单元。

4.5.3 反向传播（BP）算法的推导

- 反向传播算法的随机梯度下降算法
 - 对于每个训练样例 d , 利用关于这个样例的误差 E_d 的梯度修改权值

$$\Delta w_{ji} = -\eta \frac{\partial E_d}{\partial w_{ji}} \quad E_d(\vec{w}) = \frac{1}{2} \sum_{k \in \text{outputs}} (t_k - o_k)^2$$

符号

- x_{ji} , 单元j的第i个输入
- w_{ji} , 与单元j的第i个输入相关联的权值
- net_j , 单元j的输入的加权和
- o_j , 单元j的输出
- t_j , 单元j的目标输出
- σ , sigmoid函数
- outputs, 输出层单元集合
- Downstream(j), 单元的直接输入中包含单元j的输出的单元的集合

$$\frac{\partial E_d}{\partial w_{ji}} = \frac{\partial E_d}{\partial net_j} \frac{\partial net_j}{\partial w_{ji}} = \frac{\partial E_d}{\partial net_j} x_{ji}, \quad \text{对于 } \frac{\partial E_d}{\partial net_j}, \text{ 考虑两种情况}$$

- 输出单元

$$\frac{\partial E_d}{\partial net_j} = \frac{\partial E_d}{\partial o_j} \frac{\partial o_j}{\partial net_j}$$

$$\frac{\partial o_j}{\partial net_j} = \frac{\partial \sigma(net_j)}{\partial net_j} = o_j(1 - o_j)$$

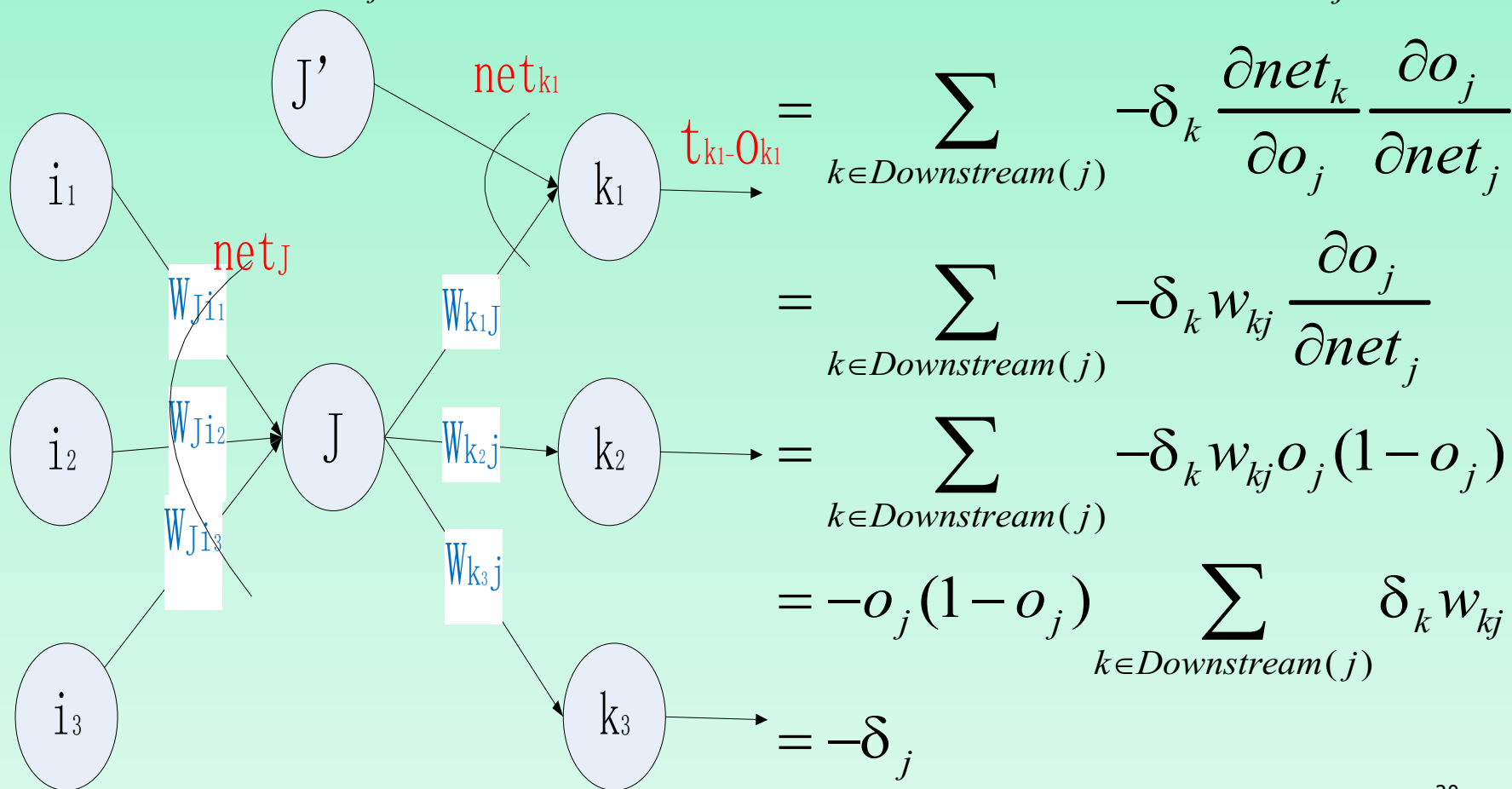
$$\begin{aligned} \frac{\partial E_d}{\partial o_j} &= \frac{\partial}{\partial o_j} \frac{1}{2} \sum_{k \in \text{outputs}} (t_k - o_k)^2 \\ &= \frac{\partial}{\partial o_j} \frac{1}{2} (t_j - o_j)^2 \\ &= \frac{1}{2} 2(t_j - o_j) \frac{\partial (t_j - o_j)}{\partial o_j} \\ &= -(t_j - o_j) \end{aligned}$$

$$\begin{aligned} \frac{\partial E_d}{\partial net_j} &= -(t_j - o_j) o_j (1 - o_j) \\ &= -\delta_j \\ \Delta w_{ji} &= -\eta \frac{\partial E_d}{\partial w_{ji}} = \eta (t_j - o_j) o_j (1 - o_j) x_{ji} \\ &= \eta \delta_j x_{ji} \end{aligned}$$

- 隐藏单元

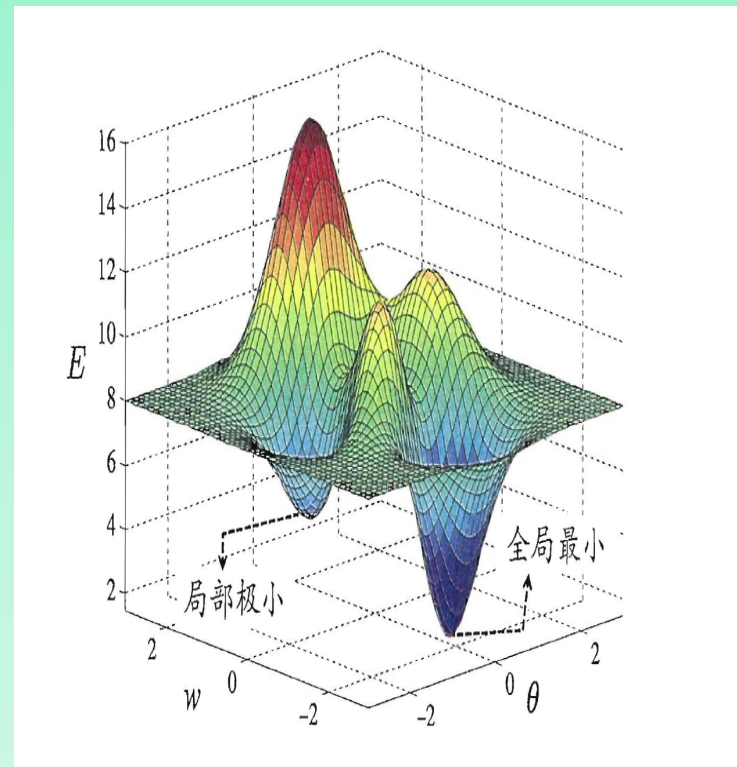
$$\frac{\partial E_d}{\partial net_j} = \sum_{k \in Downstream(j)} \frac{\partial E_d}{\partial net_k} \frac{\partial net_k}{\partial net_j}$$

$$\Delta w_{ji} = -\eta \frac{\partial E_d}{\partial w_{ji}} = \eta \delta_j x_{ji} = \sum_{k \in Downstream(j)} -\delta_k \frac{\partial net_k}{\partial net_j}$$



4.6.1 收敛性和局部极小值

- 反向传播算法仅保证收敛到误差 E 的某个局部极小值
- 误差曲面可能含有多个不同的局部极小值
- 梯度下降可能陷入这些局部极小值中的任何一个



- 为什么反向传播算法在实践中能学习到较佳的权值
 - 大量的网络权值为逃避局部极小值提供了“逃逸路线”
 - 网络权值的演变方式
 - 在初始阶段, 权值初始化接近于0, 网络表现出一个平滑的函数(近似一个线性函数)
 - 随着权值的增长, 网络会达到可以表示高度非线性网络函数的程度(权空间区域存在更多的局部极小值)
 - 网络搜索到这一点时, 已足够靠近全局最小值, 即便是局部极小值也是可以接受的

- 缓解局部极小值问题的启发式规则：
 - 添加动量项
$$\Delta w_{ji}(n) = \eta \delta_j x_{ji} + \alpha \Delta w_{ji}(n-1)$$
 - 使用随机梯度而不是真实梯度下降
 - 训练多个网络，选择最佳网络（性能在验证集上评估）

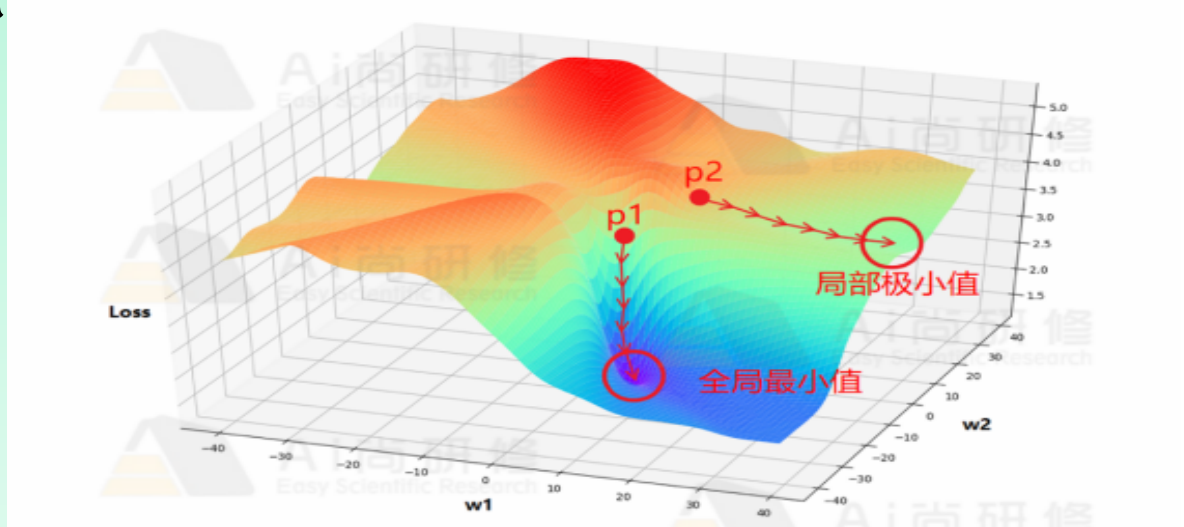
4.6.2 前馈网络的表征能力

- 布尔函数: 使用两层单元, 表示任意布尔函数
-

— 表示任意布尔函数的通用方案:

- 设计隐藏层, 对于任意输入仅有一个隐藏层单元被激活
- 设计输出层, 输出单元设计为一个仅由所希望输入模式激活的或门 (OR gate)

- 连续函数:
 - 两层单元网络: 隐藏层sigmoid单元+ 输出层线性单元
 - 任意小的误差
- 任意函数:
 - 三层单元的网络(两层sigmoid单元+ 一个线性输出层)
- 注意: 梯度下降是从一个初始的权值开始的, 因此搜索范围中的网络权向量可能不包含所有的权向量。

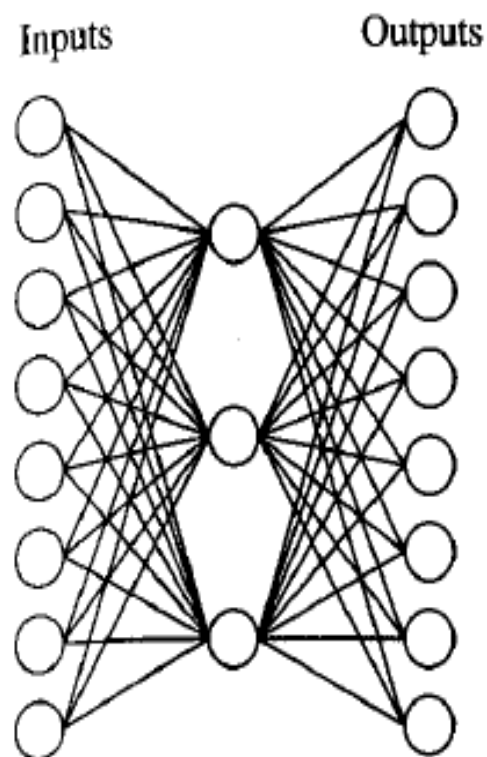


4.6.3 假设空间搜索和归纳偏置

- 反向传播算法的假设空间
 - n 维欧氏空间、连续空间
 - E 是可微的
 - 因此，梯度下降算法可以组织空间搜索以获得最佳的 h
- 反向传播算法的归纳偏置
 - 依赖于
 - 1. 梯度下降搜索 和
 - 2. 权空间覆盖可表征函数空间的方式的相互作用性
 - 粗略地刻画为在数据点之间平滑插值

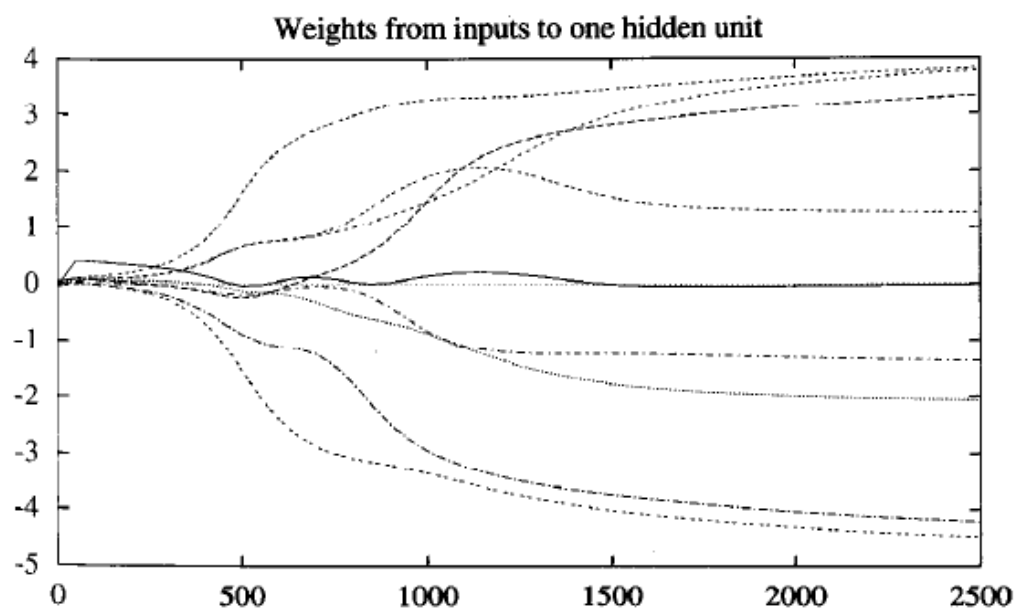
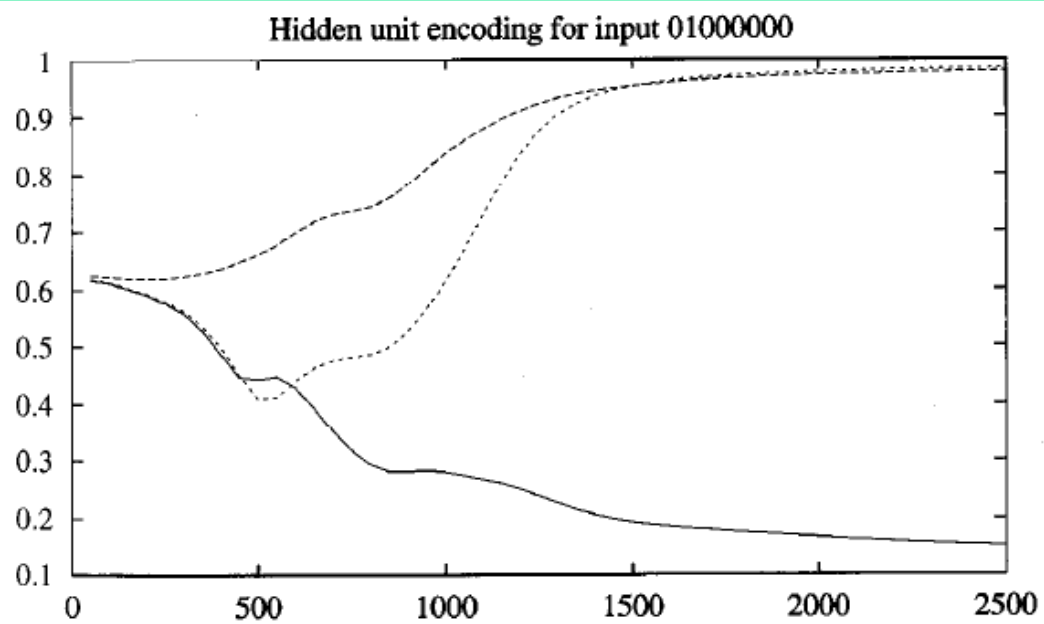
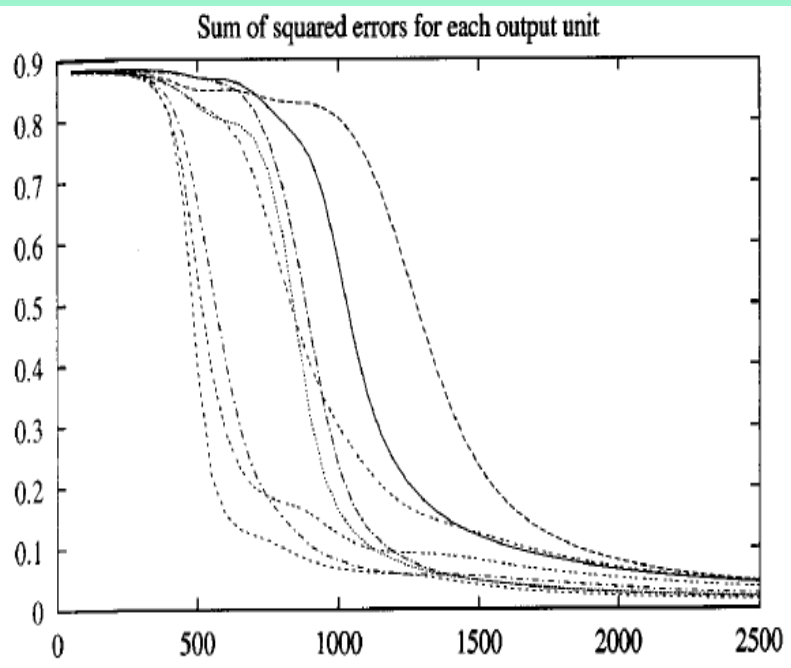
4.6.4 隐藏层表示

- BP算法能学习到有用的隐藏层中间表示（新特征表示）
 - 学习新的隐藏层特征
 - 在输入中没有明确表示
 - 捕捉输入实例中与学习目标函数最相关的特征
 - 当使用更多的网络层时，会发现更复杂特征
 - 自动发现有用的特征表示是ANN学习的关键特性
 - 允许学习算法学习出设计者没有明确引入的特征



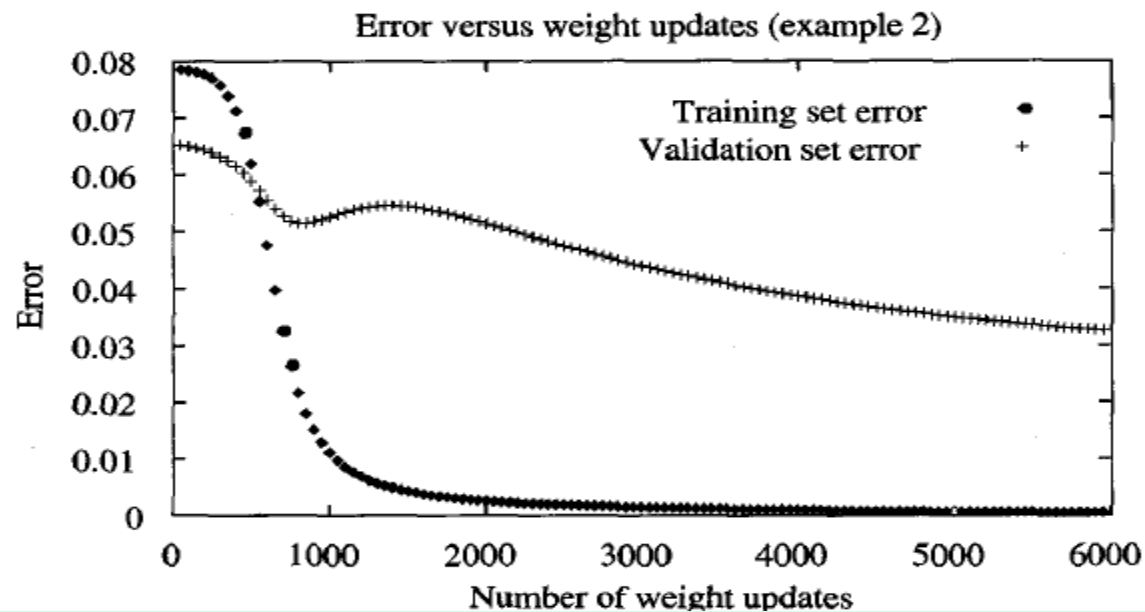
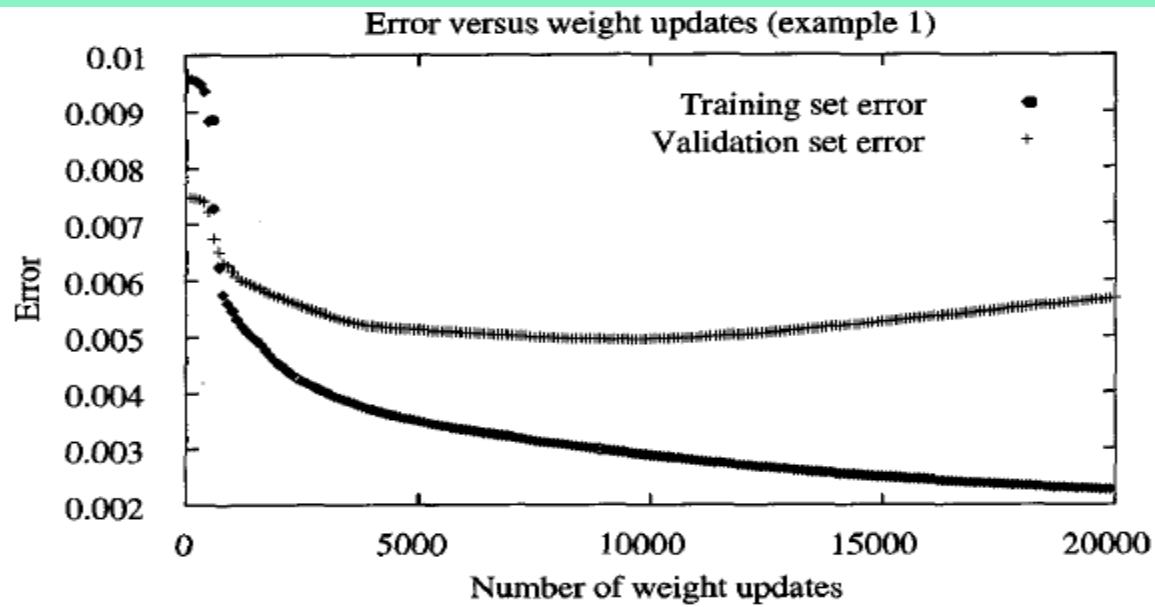
Input		Hidden Values				Output
10000000	→	.89	.04	.08	→	10000000
01000000	→	.15	.99	.99	→	01000000
00100000	→	.01	.97	.27	→	00100000
00010000	→	.99	.97	.71	→	00010000
00001000	→	.03	.05	.02	→	00001000
00000100	→	.01	.11	.88	→	00000100
00000010	→	.80	.01	.98	→	00000010
00000001	→	.60	.94	.01	→	00000001

FIGURE 4.7

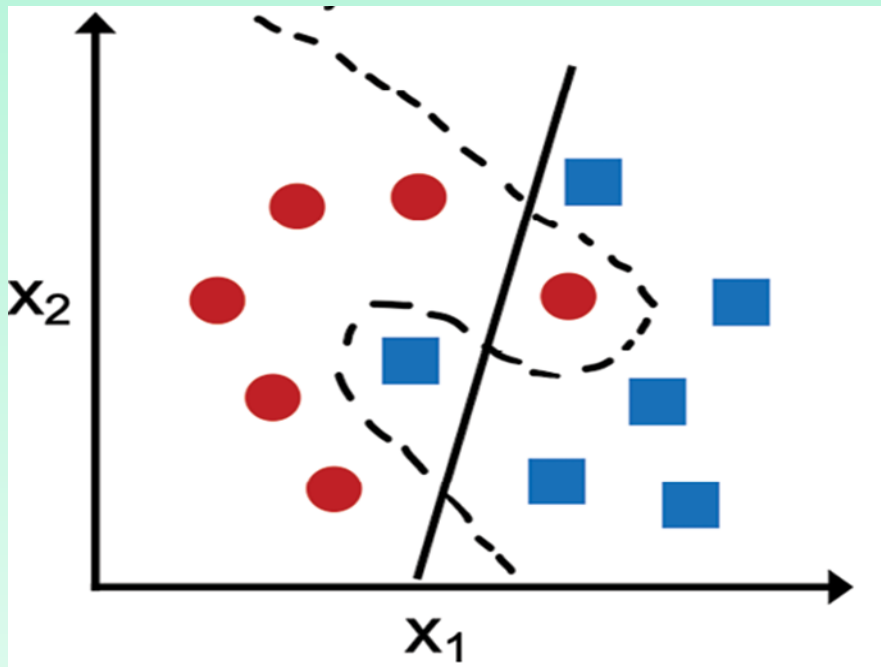


4.6.5 泛化、过度拟合和停止判据

- 终止权值更新的条件
 - 一种选择: 当误差 E 小于设置的某个阈值
 - 问题: 过度拟合
- 泛化精度
 - 网络拟合训练数据外的实例的精度
- 过拟合(如图, 见下一页)
 - 权值拟合了训练样例的“特异性” (idiosyncrasy)
 - 大量权值参数为拟合这样的“特异性”提供了很大的自由度



- 为什么过度拟合往往发生在迭代的后期？
 - 权值被初始化为较小的随机值，只能描述光滑的决策面
 - 随着训练的进行，权值开始增加，学习到的决策面的复杂性也会增加
 - 权值调整迭代次数足够多，将会产生过于复杂的决策面以拟合噪声或没有代表性的特征



- 解决过度拟合问题

- 权值衰减

- 在每次迭代过程中以某个小因子降低每个权值
 - 等效于修改E的定义，加入一个与网络权值的总量相应的惩罚项。

$$E(\vec{w}) = \frac{1}{2} \sum_{d \in D} \sum_{k \in \text{outputs}} (t_{kd} - o_{kd})^2 + \gamma \sum_{i,j} w_{ji}^2$$

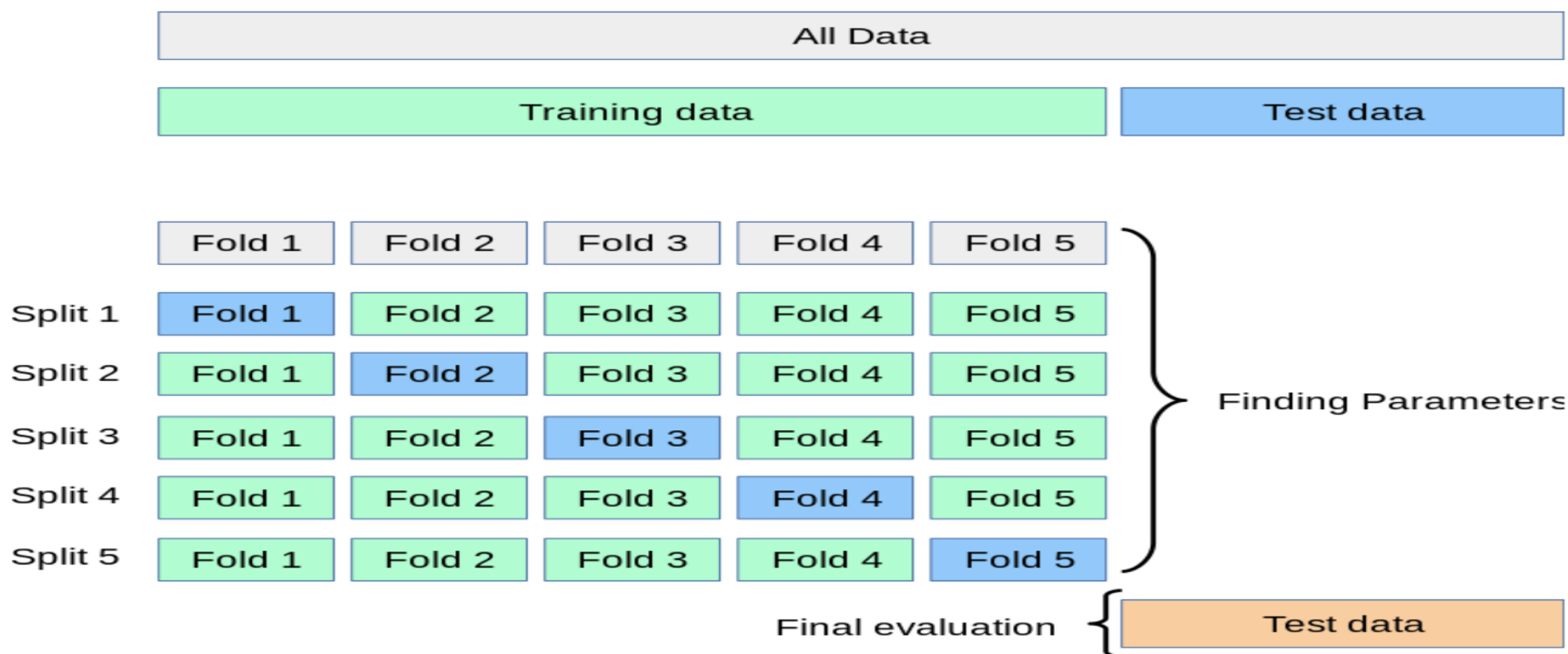
- 其动机是保持权值较小，使学习过程向着复杂的决策面的反方向偏置（请参见第4.8.1节）。

- 交叉验证方法

- 使用训练集驱动梯度下降搜索
 - 使用验证集监视误差
 - 需要更多的训练样例

– k-fold交叉验证

- 用于避免小训练集的过拟合。
- m 个样例被划分为 k 个不相交的子集，每个子集的大小为 m / k 。
- 运行 k 次，每次用一个子集作为验证集并合并其它子集作为训练集合
- 交叉验证过程确定在验证集合上取得最佳性能的迭代次数 $\bar{i}$ ，计算均值 $\bar{i}$
- 最后运行一次反向传播算法，迭代 $\bar{i}$ 次



4.7 举例：人脸识别

- 训练数据
 - 20个不同人的摄影图像
 - 每人32张
 - 不同表情
 - 快乐、伤心、愤怒、中性
 - 面部方向
 - 左、右、正前、上
 - 其他
 - 是否带太阳镜, 人后面的背景, 穿着
 - 共收集624灰度图像
 - 分辨率 120x128
 - 色素: 0 --- 255
- 任务: 学习图像中人脸的朝向



4.7.2 设计要素

- 输入编码
 - 表示形式1：提取边缘，亮度一致的区域或其他局部图像特征。
 - 问题：导致每幅图像有不同数量的特征参数，而ANN具有固定数量的输入单元
 - 表示形式2：每个像素对应一个网络输入
 - 输入数量固定
 - 像素灰度值线性缩放到0到1的范围内（使网络输入值与隐藏单元和输出单元的取值范围相同）



4.7.2 设计要素

- 输出编码
 - 使用单一的输出单元
 - 使用四个不同的输出单元
 - 取具有最高值的输出作为网络的预测值，这种方法被称为n取1（1-of-n）输出编码
- 为什么选择1-of-n输出编码
 - 提供更大的自由度来表示目标函数
 - 提供网络预测的置信度

- 4、个输出单元的目标值
 - 一个显而易见的选择 $\langle 1, 0, 0, 0 \rangle \dots$
 - 使用0.1 and 0.9, $\langle 0.9, 0.1, 0.1, 0.1 \rangle \dots$
 - 避免使用0和1的原因
 - Sigmoid 单元不能产生这样的输出
 - 避免权值无限增长
 - Sigmoid 单元在有限权值情况下可达到0.1和0.9

- 网络结构图
 - 网络包含多少个单元以及如何互连
 - 分层网络，一层的每个单元连接到下一层的每个单元
 - 两层sigmoid单元
 - 隐藏单元的数量
 - 3个神经元, 90%的测试精度, Sun SParc5工作站上训练5分钟, 260 幅训练图像
 - 30个神经元, 提高一到两个百分点, 训练接近一小时
 - 隐藏单元的数量
 - 需要最少数量的隐藏单位
 - 多余的隐藏单元不会显著提高泛化精度
 - 过度拟合：增加隐藏单元的数量通常会增加过度拟合训练数据的风险

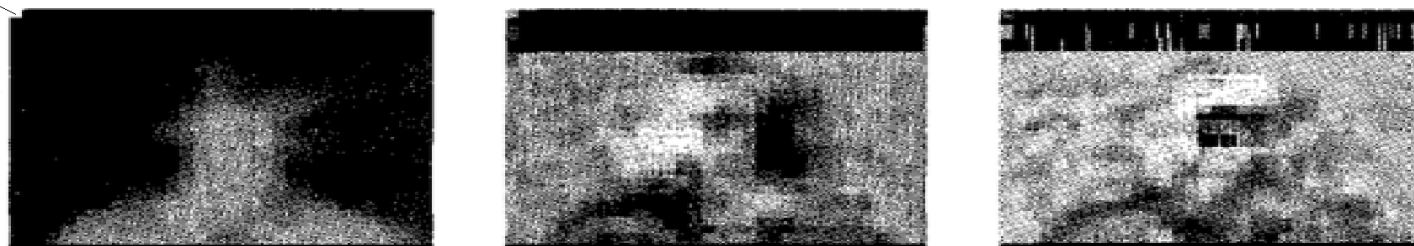
- 学习算法的其他参数
 - 学习速率:0.3, 冲量 : 0.3
 - 赋予更低的值产生大体相当的泛化精度, 但训练时间较长
 - 这两个值过高, 训练将不能收敛
 - 使用完全的梯度下降
 - 输出单元的权值初始化为较小的随机值
 - 输入单元的权值初始化为零
 - 训练迭代次数:
 - 由训练集和独立的验证集决定
 - 最终网络:
 - 在验证集合上, 精度最高的网络
 - 最终精度:
 - 在第三个集合—测试集合上测量得到

4.7.3 学习到的隐藏层表示

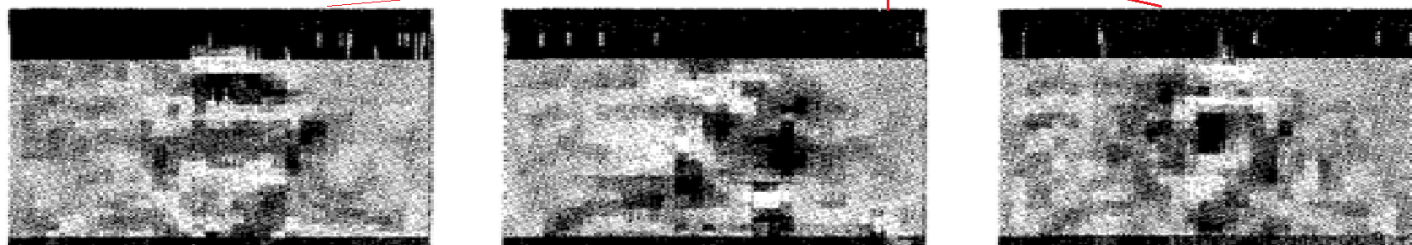
- 1次迭代后输出权值
 - 1次迭代后，从输入单元到隐藏单元的权值（请参阅下一页）
- 100次迭代后输出权值
 - 100次迭代后，从输入单元到隐藏单元的权值（请参阅下一页）



30 × 32 resolution input images



Network weights after 1 iteration through each training example



Network weights after 100 iterations through each training example

4.8.1 其他可选的误差参数

- 增加一个惩罚项
 - 搜寻较小的权值向量，减少过度拟合风险

$$E(\vec{w}) = \frac{1}{2} \sum_{d \in D} \sum_{k \in \text{outputs}} (t_{kd} - o_{kd})^2 + \gamma \sum_{i,j} w_{ji}^2$$

- 对误差增加一项目标函数的斜率（slop）或导数

$$E(\vec{w}) = \frac{1}{2} \sum_{d \in D} \sum_{k \in \text{outputs}} \left[(t_{kd} - o_{kd})^2 + \mu \sum_{j \in \text{inputs}} \left(\frac{\partial t_{kd}}{\partial x_d^j} - \frac{\partial o_{kd}}{\partial x_d^j} \right)^2 \right]$$

- 使网络对目标值的交叉熵(cross entropy)最小化
 - 用于学习概率函数（属于1或0的概率）

交叉熵：
$$-\sum_{d \in D} t_d \log o_d + (1 - t_d) \log(1 - o_d)$$

- 第6章讨论为什么最可能的假设就是使交叉熵最小化的假设
- 多分类交叉熵损失函数：（使用softmax函数来找到概率 P_{ij} ，后面讲）

$$L(X_i, Y_i) = - \sum_{j=1}^c y_{ij} * \log(p_{ij})$$

where Y_i is one-hot encoded target vector $(y_{i1}, y_{i2}, \dots, y_{ic})$,

$$y_{ij} = \begin{cases} 1, & \text{if } i_{th} \text{ element is in class } j \\ 0, & \text{otherwise} \end{cases}$$

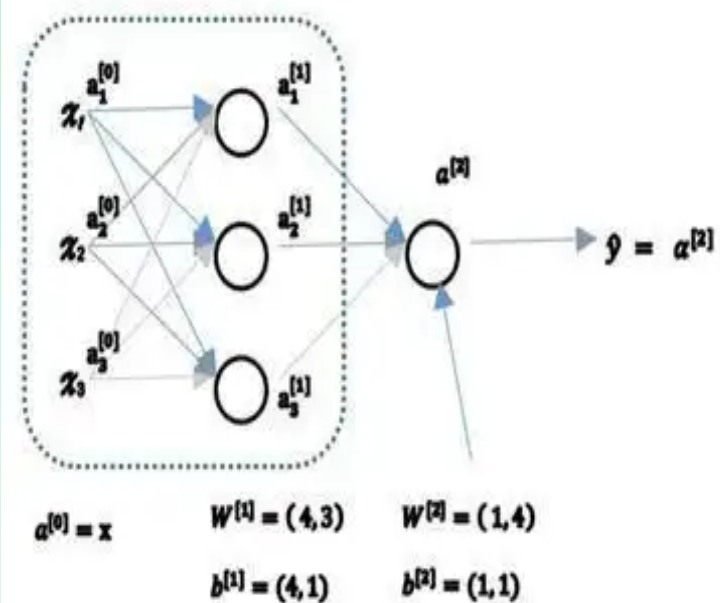
$p_{ij} = f(X_i) = \text{Probability that } i_{th} \text{ element is in class } j$

• 神经网络的矩阵表示

- 简化表达式：复杂的网络结构可通过矩阵乘法直观表示
- 并行计算：矩阵运算天然适配GPU等硬件的并行处理特性，可大幅提升训练和推理效率

$$a^{[1]} = \begin{bmatrix} a_1^{[1]} \\ a_2^{[1]} \\ a_3^{[1]} \\ a_4^{[1]} \end{bmatrix} = \sigma(z^{[1]})$$

input layer hidden layer output layer



$$\begin{aligned} z^{[1]} &= W^{[1]}x + b^{[1]} \\ a^{[1]} &= \sigma(z^{[1]}) \\ z^{[2]} &= W^{[2]}a^{[1]} + b^{[2]} \\ a^{[2]} &= \sigma(z^{[2]}) \end{aligned}$$

$$\begin{bmatrix} z_1^{[1]} \\ z_2^{[1]} \\ z_3^{[1]} \\ z_4^{[1]} \end{bmatrix} = \begin{bmatrix} \dots W_1^{[1]T} \dots \\ \dots W_2^{[1]T} \dots \\ \dots W_3^{[1]T} \dots \\ \dots W_4^{[1]T} \dots \end{bmatrix} * \begin{bmatrix} x_1 \\ x_2 \\ x_3 \end{bmatrix} + \begin{bmatrix} b_1^{[1]} \\ b_2^{[1]} \\ b_3^{[1]} \\ b_4^{[1]} \end{bmatrix}$$

$$a^{[n]} = \sigma(z^{[n]})$$

$$z^{[n]} = w^{[n]}x + b^{[n]}$$



4.9 小结和补充读物

- 通过连续或离散值属性学习实值和向量值函数。
- 对训练数据中的噪声具有很好的健壮性。
- 反向传播算法是最常见的网络学习算法。
- 反向传播算法考虑的假设空间是带权神经网络所能表示的所有函数的空间。
- 包含3层单元的前馈网络能够以任意精度逼近任意函数
- 反向传播算法使用梯度下降搜索假设空间，迭代减小网络误差以拟合训练数据。

- 梯度下降收敛到局部极小值。
- 梯度下降算法应用广泛，可用于搜索连续参数的假设空间，其中训练误差是关于假设参数的可微函数。
- BP可以学习到网络输入中未明确出现的新特征。
- 过度拟合和交叉验证。

- 作业:4.2,4.3, (4.5,4.7选做)